Speaker notes

LLM-Guided Grammar-Aware Fuzzing of PostgreSQL · CS5187 selected project presentation · Tue 10 Nov 2026, Room 5G4

SPEAKER	SLIDES	TIME
Venu	1 Title · 2 Target · 3 How our fuzzer works · 6 first half (Bug 5, JSON_TABLE)	≈ 3:20
Gowtham	4 Bugs 1–2 (#19803, to_char) · 7 All nine bugs · 8 Thank you	≈ 2:20
Bhuvan	5 Bugs 3–4 (statistics, NOT IN) · 6 second half (Bug 6, IGNORE NULLS)	≈ 2:15
Total		**≈ 7:45**

Say it in your own words; do not read the slide. On each bug slide point at the code box and say the one line in the "expected / got" comment. If time is short, skip the "Fix" line: the badge already says it is fixed.

---

1 Title

Venu · 0:25

Good afternoon. I am Venu, and with me are Bhuvan and Gowtham. Our project is LLM-guided, grammar-aware fuzzing of PostgreSQL. A fuzzer is a program that automatically writes a huge number of test queries and checks whether the database handles them correctly. Ours found nine bugs in PostgreSQL. Six of them are already fixed upstream, and for five of those six the fix that went in is a patch we wrote. In the next seven minutes we show the target, the techniques, and our top six bugs.

---

2 Our target: PostgreSQL

Venu · 0:50

Our target is PostgreSQL, one of the most widely used open-source databases. Its query planner, the part that decides how a query is executed, changes almost every week, and new code is the least tested code. So we tested the development branch, master, built with PostgreSQL's internal assertion checks turned on, and we compared it with the last release, 18. We treat four kinds of failure as bugs, all equally: a crash or failed assertion, a wrong result, an internal error on a valid query, and a query that never stops. The bottom line: nine bugs reported, six fixed, and five of the fixes are patches we wrote.

3 How our fuzzer works

Venu · 1:20

This is the whole tool in one picture. Step one: an LLM reads recent PostgreSQL commits and proposes new grammar rules for the SQL features those commits touch; a gate keeps a rule only if the server actually runs it. Step two: we generate queries from a typed SQL grammar, 78 rules and 421 alternatives, so almost every query is valid and reaches the planner instead of failing in the parser. Step three: every query runs on master with assertions on, under 18 different planner settings, and also on release 18.6. Step four: the checks below decide whether something is wrong. A crash check. A metamorphic rewrite check: for example we split a WHERE condition into its true, false and NULL parts, and the parts must add up to the original answer. Plan variants: the same query under different planner settings must return the same rows. Cross-version: master and release 18 must agree. And a hang check: a query must stop at the timeout. Step five: a failing query is shrunk to a few lines and turned into a draft bug report. We also wrote small targeted fuzzers for new features, for example extended statistics.

4 Bugs 1–2: a planner crash and a broken to_char()

Gowtham · 1:30

I am Gowtham; I will take the first two bugs. Bug one is a crash in the planner. The planner prepares a parallel plan for a table, then later decides the table is not safe for parallel execution, but forgets to throw away the plan it already made. PostgreSQL's internal assertion catches this and the server stops. This one came from the LLM part of our fuzzer: the LLM read a recent planner commit and added grammar rules for those features, and a query generated from them crashed the server. David Rowley fixed it and back-patched the fix to version 14. Bug two is about to_char, the function that formats numbers as text. If the format ends with a decimal point, PostgreSQL copies the invisible end-of-string byte into the result, so everything after it, here the plus sign and the word items, silently disappears. We found it with the cross-version check: we ran thousands of random to_char calls on master and on release 18 and compared the answers. We wrote the fix, and Tom Lane committed it for version 19.

5 Bugs 3–4: a statistics crash and a wrong NOT IN

Bhuvan · 1:30

I am Bhuvan. Bug three: PostgreSQL can keep extra statistics on expressions to estimate how many rows a query returns. If the same condition appears twice in the WHERE clause, an assertion in that estimate code fails and the server crashes. Our extended-statistics fuzzer found it by creating random statistics objects and random WHERE clauses that reuse the same expressions. Our patch was committed by David Rowley and back-patched to version 14. Bug four is a wrong result. Version 19 added a speed-up that turns NOT IN into an anti-join. That is only correct if no value can be NULL. But an implicit cast can turn a value into NULL: here zero becomes NULL. And 1 NOT IN (NULL) is unknown, so the correct answer is no rows, but master returns the row. Our differential checks caught it, because release 18, which does not have the speed-up, returns no rows. Richard Guo committed our patch for version 19.

6 Bugs 5–6: JSON_TABLE and a query that never stops

Venu → Bhuvan · 1:30

Venu Bug five is in JSON_TABLE, which turns JSON into rows. PostgreSQL forgot to assign collations, the text comparison rules, to JSON_TABLE's input. As a result a row comparison inside it compares zero columns. Row one-two less than row three-four should be true; versions 17 and 18 say false, and master crashes on an assertion. Our grammar fuzzer found the crash on master, and running the same query on release 18 showed the silent wrong answer. Amit Langote committed our patch this week and back-patched it to version 17.

Bhuvan Bug six is a hang. first_value and nth_value with the new IGNORE NULLS option can loop forever on this window frame. Worse, the query ignores statement_timeout and cancel requests, so one query can hold a CPU until the process is killed. Our hang check flagged it because the query ran past its timeout and could not be cancelled. Tatsuo Ishii committed our patch for version 19.

7 All nine bugs

Gowtham · 0:40

Here are all nine bugs. The six we just showed are fixed upstream, five of them with our own patches. They cover every kind of failure: crashes, wrong results, an internal error and a hang. Of the other three, the MERGE internal error is confirmed by committer Tender Wang, who posted a fix; the range bug and the parallel EXPLAIN crash are reported, and for the range bug we also sent a patch. The main lesson: writing valid SQL is the easy part; knowing the right answer is the hard part, and the checks we built are what turned generated queries into real bug reports.

8 Thank you

Gowtham · 0:10

Thank you. Everything, the fuzzer, the reproducers and our patches, is in our repository. We are happy to take questions.